

Attention II: The Math

Queries, keys and values, one matrix at a time · video 2:18

This lesson has one big idea: all of attention, for every token at once, is **a handful of matrix multiplications**. Project the tokens to queries, keys and values, score every query against every key, scale, mask the future, softmax each row, and mix the values: $\text{softmax}(\mathbf{Q} \mathbf{K}^T / \sqrt{d}) \cdot \mathbf{V}$.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 10).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. From X to Q, K and V	▶ 0:08-0:42	3	☐
2. Scores: Q times K transposed	▶ 0:42-0:59	4	☐
3. Scaling by $\sqrt{d}$	▶ 0:59-1:09	4	☐
4. The causal mask	▶ 1:09-1:20	4	☐
5. Softmax, row by row	▶ 1:20-1:31	4	☐
6. Mixing the values	▶ 1:31-1:43	4	☐
7. The whole formula, in NumPy	▶ 1:43-2:16	3	☐

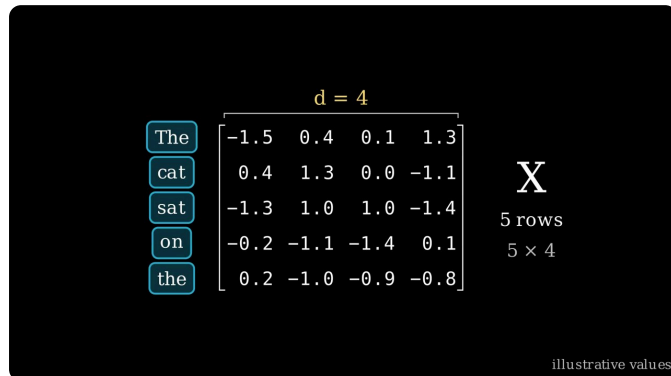
Before you start: this lesson puts episode 5 (queries, keys, values, weights that add up to 1, only looking backwards) into numbers. It uses the dot product (Foundations F02), matrix multiplication (F03), e^x (F05) and softmax (F07); F08 explains why scores grow with d . The code in concept 7 uses NumPy (F11).

CONCEPT 1 OF 7

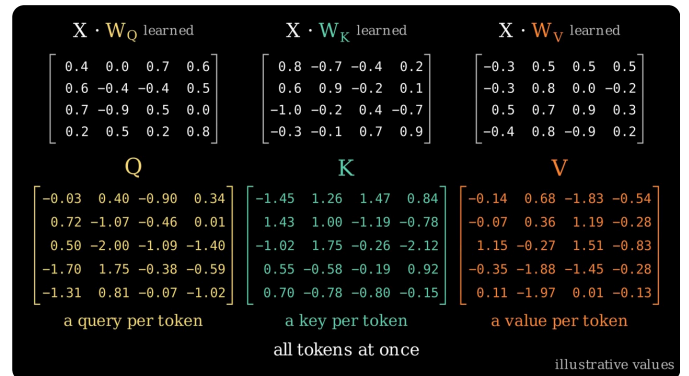
From X to Q, K and V

▶ Video 0:08–0:42

Now with actual numbers. The five tokens *The cat sat on the* are stacked as the rows of one matrix, **X**: 5 rows (one per token) and **d** columns. The video uses $d = 4$, so every number fits on screen.



▶ 0:29 X: one row per token, $d = 4$ columns.



▶ 0:42 Q, K and V: a query, key and value for every token.

Multiply **X** by three learned 4×4 matrices: $\mathbf{Q} = \mathbf{X} \cdot \mathbf{W}_Q$, $\mathbf{K} = \mathbf{X} \cdot \mathbf{W}_K$, $\mathbf{V} = \mathbf{X} \cdot \mathbf{W}_V$. Row i of **Q** is token i 's query, row i of **K** its key, row i of **V** its value: every token, all at once, in three matrix multiplications (Foundations F03).

KEY IDEA

X ($n \times d$) times learned $d \times d$ matrices gives **Q**, **K** and **V** (each $n \times d$). **One row per token**: its query, its key, its value.

Check 1: From X to Q, K and V

Answer all 3 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

X is 5×4 and $\mathbf{W}_Q$ is 4×4 . What is the shape of **Q**? What would it be for a text of 12 tokens?

1.2 CALCULATION

The first row of **X** (*The*) is $(-1.5, 0.4, 0.1, 1.3)$ and the first column of $\mathbf{W}_Q$ is $(0.4, 0.6, 0.7, 0.2)$. Compute the first number of *The*'s query.

1.3 TRUE OR FALSE

"A longer text needs a bigger $\mathbf{W}_Q$."

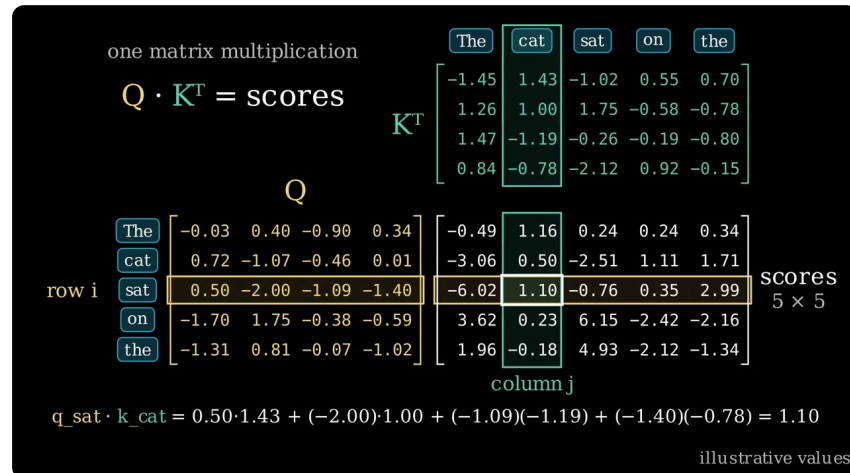
True ☐ False ☐

CONCEPT 2 OF 7

Scores: Q times K transposed

▶ Video 0:42-0:59

Next, compare every query with every key. That is a **single matrix multiplication**: $\text{scores} = Q \cdot K^T$. Transposing K turns its rows (the keys) into columns, so entry (i, j) is row i of Q times column j of K^T , the dot product $q_i \cdot k_j$. The result is a 5×5 grid: row i, column j says how well token i's question matches token j's key.



▶ 0:58 $Q \cdot K^T$: the 5×5 grid of scores. Row sat, column cat is $q_{\text{sat}} \cdot k_{\text{cat}} = 1.10$.

KEY IDEA

$\text{scores}[i, j] = q_i \cdot k_j$. $Q \cdot K^T$ computes all $n \times n$ query-key dot products in one go.

Check 2: Scores: Q times K transposed

Answer all 4 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

Check the video's example: $q_{\text{sat}} = (0.50, -2.00, -1.09, -1.40)$ and $k_{\text{cat}} = (1.43, 1.00, -1.19, -0.78)$. Compute $q_{\text{sat}} \cdot k_{\text{cat}}$ to two decimals.

2.2 CALCULATION

$q_{\text{The}} = (-0.03, 0.40, -0.90, 0.34)$ and $k_{\text{The}} = (-1.45, 1.26, 1.47, 0.84)$. Compute the top-left score of the grid.

2.3 MULTIPLE CHOICE

A text has 100 tokens and $d = 4$. What is the shape of $Q \cdot K^T$?

- A. 100×4
- B. 4×4
- C. 100×100
- D. 4×100

2.4 TRUE OR FALSE

"The score of sat looking at cat equals the score of cat looking at sat."

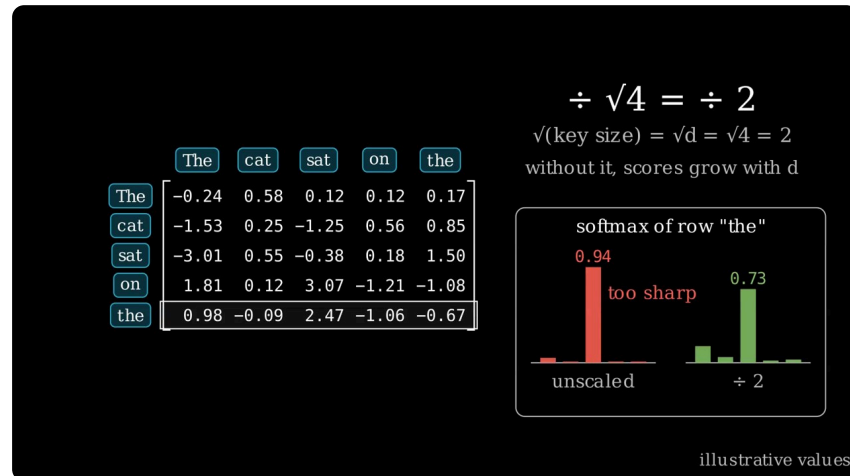
True ☐ False ☐

CONCEPT 3 OF 7

Scaling by $\sqrt{d}$

▶ Video 0:59-1:09

Then divide every score by $\sqrt{d}$, the square root of the key size. Here $d = 4$, so we divide by 2. Without this, scores grow with the vector length (F08), the softmax gets **too extreme**, and training struggles. In the inset, row *the* turned straight into weights gives the top token 0.94, almost everything; after $\div 2$ it gets 0.73 and the others keep a share.



▶ 1:09 Every score $\div \sqrt{4} = \div 2$. Inset: row "the" turned into weights, without and with the scaling.

KEY IDEA

scaled scores = $Q \cdot K^T / \sqrt{d}$. This keeps the scores moderate, so the softmax is not too sharp.

Check 3: Scaling by $\sqrt{d}$

Answer all 4 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

sat's score for cat is 1.104. What is it after scaling ($d = 4$)?

.....

3.2 CALCULATION

A model uses keys of size 64. What number are its scores divided by?

.....

3.3 MULTIPLE CHOICE

Why divide the scores by $\sqrt{d}$?

- A. So that every row of scores adds up to 1
- B. Because scores grow with the vector length, which makes the softmax too extreme
- C. To turn negative scores into positive ones
- D. To stop tokens from seeing the future

3.4 TRUE OR FALSE

"Dividing a row of scores by 2 can change which token gets the largest weight."

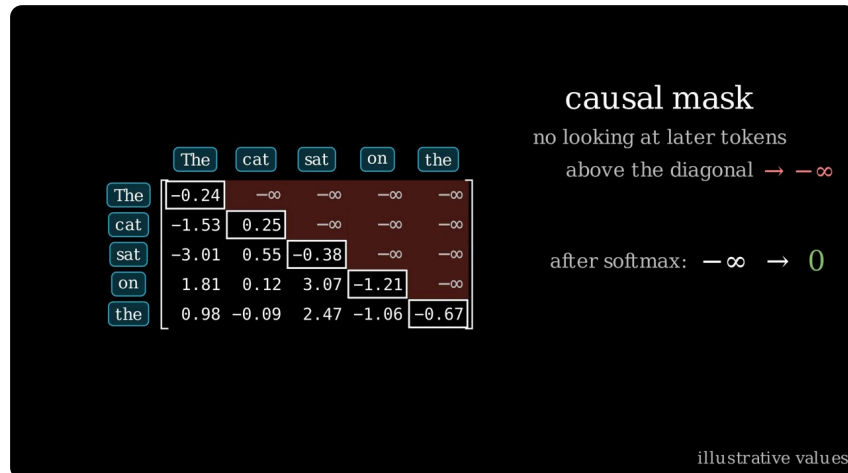
True ☐ False ☐

CONCEPT 4 OF 7

The causal mask

▶ Video 1:09-1:20

Now the **causal mask**: episode 5's “only look backwards” rule, in numbers. A token must not see the future, so every score **above the diagonal** (column j later than row i) is set to $-\infty$. After the next step, the softmax, those become **exactly zero**, because $e^{-\infty} = 0$.



▶ 1:20 Every score above the diagonal becomes $-\infty$; after the softmax it will be exactly 0.

KEY IDEA

Mask: **scores[i, j] = $-\infty$ whenever $j > i$** . After the softmax, later tokens get weight exactly 0.

Check 4: The causal mask

Answer all 4 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

How many entries of the 5×5 grid are masked? How many would be masked for a 100-token text?

4.2 MULTIPLE CHOICE

Which of these scores gets masked?

- A. row sat, column cat
- B. row cat, column sat
- C. row the, column The
- D. row on, column on

4.3 SHORT ANSWER

Which row of the grid has no masked entries at all? Why?

4.4 TRUE OR FALSE

“Setting the masked scores to 0 instead of $-\infty$ would work just as well.”

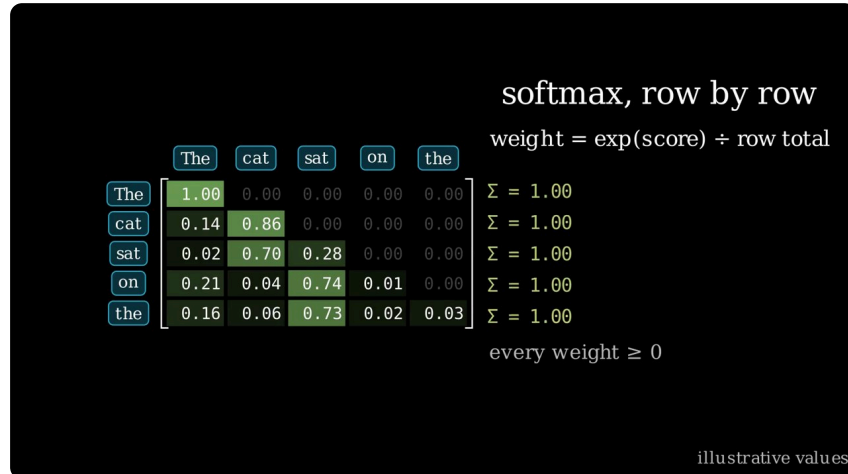
True ☐ False ☐

CONCEPT 5 OF 7

Softmax, row by row

▶ Video 1:20-1:31

The **softmax** (F07) turns each row into weights: **exponentiate every score, then divide by the row's total**. Every row is now positive (masked entries exactly 0) and **adds up to one**: episode 5's budget, now computed. Row *sat*, for example, becomes 0.02, 0.70, 0.28: *sat* listens mostly to *cat*.



▶ 1:30 The attention weights: every row is ≥ 0 and adds up to 1.00.

KEY IDEA

weight = $\exp(\text{score}) \div \text{row total}$, one row at a time. Each row is all ≥ 0 and adds up to 1.

Check 5: Softmax, row by row

Answer all 4 questions before looking at the answers. Pass mark: all correct.

5.1 CALCULATION

Row *cat* has scaled scores -1.53 and 0.25 (the rest is masked). Compute its two weights. Use $e^{-1.53} \approx 0.217$ and $e^{0.25} \approx 1.284$.

5.2 SHORT ANSWER

Why is row *The* exactly 1.00, 0, 0, 0, 0, whatever its score is?

5.3 MULTIPLE CHOICE

You add 1 to every unmasked score in a row. What happens to that row's weights?

- A. They all grow
- B. They stay exactly the same
- C. The largest grows and the others shrink
- D. They no longer add up to 1

5.4 TRUE OR FALSE

"A negative score gives a negative weight."

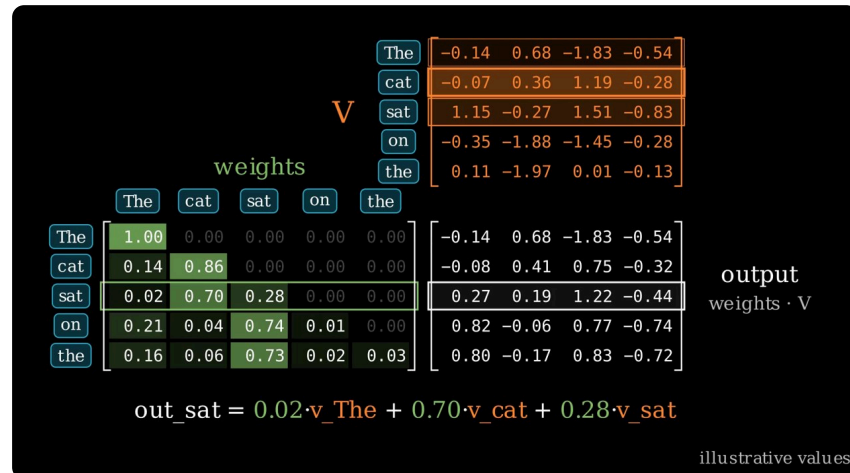
True ☐ False ☐

CONCEPT 6 OF 7

Mixing the values

▶ Video 1:31-1:43

Finally, **multiply the weights by V**. Each token's output row is a **weighted blend of the values it attends to**: $\text{out}_{\text{sat}} = 0.02 \cdot v_{\text{The}} + 0.70 \cdot v_{\text{cat}} + 0.28 \cdot v_{\text{sat}}$. The first token can only see itself (weight 1.00), so it simply gets its own value back.



▶ 1:37 output = weights · V. Row sat is $0.02 \cdot v_{\text{The}} + 0.70 \cdot v_{\text{cat}} + 0.28 \cdot v_{\text{sat}}$.

KEY IDEA

output = weights · V. Row i is the weighted sum of the values of token i and the tokens before it.

Check 6: Mixing the values

Answer all 4 questions before looking at the answers. Pass mark: all correct.

6.1 CALCULATION

The first numbers of v_{The} , v_{cat} and v_{sat} are -0.14 , -0.07 and 1.15 . With the weights 0.02 , 0.70 and 0.28 , compute the first number of out_{sat} .

6.2 MULTIPLE CHOICE

The weights are 5×5 and V is 5×4 . What is the shape of the output?

- A. 5×5
- B. 5×4
- C. 4×4
- D. 4×5

6.3 SHORT ANSWER

Row *on* has weights 0.21 , 0.04 , 0.74 , 0.01 , 0 . Whose value dominates out_{on} , and whose value is not in it at all?

6.4 TRUE OR FALSE

" out_{The} equals v_{The} ."

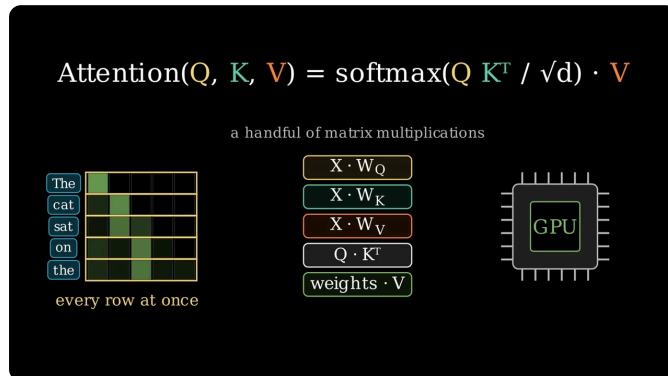
True ☐ False ☐

CONCEPT 7 OF 7

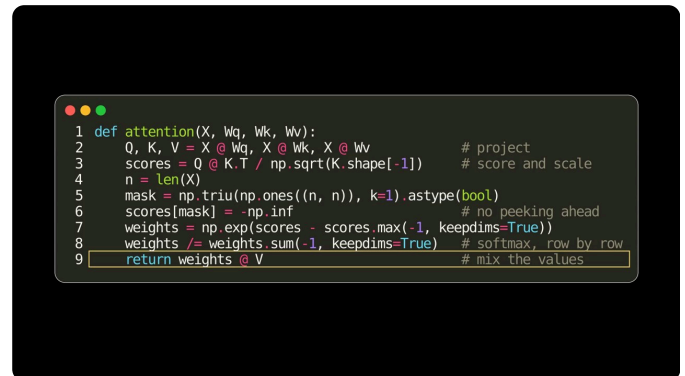
The whole formula, in NumPy

▶ Video 1:43-2:16

Put together, it is one line: **Attention(Q, K, V) = softmax(Q K^T / √d) · V**, with the causal mask applied before the softmax. Every token is handled **in parallel**, in a handful of matrix multiplications. That is a big reason transformers run so well on GPUs.



▶ 1:59 One line, a handful of matrix multiplications, every token in parallel.



▶ 2:06 The same thing in NumPy: project, score and scale, mask, softmax, mix.

```
def attention(X, Wq, Wk, Wv):
    Q, K, V = X @ Wq, X @ Wk, X @ Wv          # project
    scores = Q @ K.T / np.sqrt(K.shape[-1])    # score and scale
    n = len(X)
    mask = np.triu(np.ones((n, n)), k=1).astype(bool)
    scores[mask] = -np.inf                      # no peeking ahead
    weights = np.exp(scores - scores.max(-1, keepdims=True))
    weights /= weights.sum(-1, keepdims=True)   # softmax, row by row
    return weights @ V                          # mix the values
```

`np.triu(..., k=1)` marks the entries above the diagonal. The softmax subtracts each row's max first, the overflow trick from F07; it does not change the weights. One attention layer asks one kind of question; episode 7 asks several at once.

KEY IDEA

softmax(Q K^T / √d) · V, plus the causal mask: all tokens at once, in a few matrix multiplications.

Check 7: The whole formula, in NumPy

Answer all 3 questions before looking at the answers. Pass mark: all correct.

7.1 PUT IN ORDER

Put the steps in the order the code runs them: *softmax · mask · multiply by V · Q · K^T · divide by √d · project X to Q, K, V*.

7.2 MULTIPLE CHOICE

Why does attention run so well on GPUs?

- A. It handles every token at once, as a few large matrix multiplications
- B. It skips most tokens, so there is little to compute
- C. It processes one token after another, in a loop
- D. It uses no multiplications, only additions

7.3 HANDS-ON CODE

Try it yourself. The code runs the video's `attention` function on 3 tokens with $d = 2$. All three weight matrices are the identity, so $Q = K = V = X$. (a) Predict row 1 of the output without computing anything. (b) By hand, compute token 2's two weights and its output row (remember to divide by $\sqrt{2}$), then run the code to check. (c) Comment out the line `scores[mask] = -np.inf` and run again. Which rows change, and why?

```
import numpy as np

def attention(X, Wq, Wk, Wv):
    Q, K, V = X @ Wq, X @ Wk, X @ Wv          # project
    scores = Q @ K.T / np.sqrt(K.shape[-1])    # score and scale
    n = len(X)
    mask = np.triu(np.ones((n, n)), k=1).astype(bool)
    scores[mask] = -np.inf                      # no peeking ahead
    weights = np.exp(scores - scores.max(-1, keepdims=True))
    weights /= weights.sum(-1, keepdims=True)   # softmax, row by row
    return weights @ V                          # mix the values

X = np.array([[1.0, 0.0],      # token 1
              [0.0, 1.0],      # token 2
              [1.0, 1.0]])     # token 3
I = np.eye(2)                  # Wq = Wk = Wv = identity: Q = K = V = X
print(attention(X, I, I, I).round(2))
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. From X to Q, K and V · rewatch ▶ 0:08-0:42

1.1 5 × 4; 12 × 4. $(n \times d) \cdot (d \times d) = n \times d$: one 4-number query per token.

1.2 -0.03. $-0.60 + 0.24 + 0.07 + 0.26 = -0.03$, the top-left entry of Q in the video.

1.3 False. W_Q is $d \times d$ whatever the text length. More tokens only means more rows in X, and so in Q, K and V.

2. Scores: Q times K transposed · rewatch ▶ 0:42-0:59

2.1 1.10. $0.715 - 2.000 + 1.297 + 1.092 = 1.104 \approx 1.10$.

2.2 -0.49. $0.044 + 0.504 - 1.323 + 0.286 = -0.490$, as in the video.

2.3 C. $(100 \times 4) \cdot (4 \times 100) = 100 \times 100$: one score for every (query token, key token) pair.

2.4 False. $\text{sat} \rightarrow \text{cat}$ is $q_{\text{sat}} \cdot k_{\text{cat}} = 1.10$, but $\text{cat} \rightarrow \text{sat}$ is $q_{\text{cat}} \cdot k_{\text{sat}} = -2.51$. Queries and keys are different vectors, so the grid is not symmetric.

3. Scaling by $\sqrt{d}$ · rewatch ▶ 0:59-1:09

3.1 0.55. $1.104 \div \sqrt{4} = 1.104 \div 2 = 0.552 \approx 0.55$, the value in the scaled grid.

3.2 8. $\sqrt{64} = 8$.

3.3 B. A is the softmax's job, D is the mask's job, and dividing by a positive number never changes a sign.

3.4 False. Dividing by a positive number keeps the order, so the top score stays on top (in row *the*, *sat* wins both times). Only the sharpness changes.

4. The causal mask · rewatch ▶ 1:09-1:20

4.1 10; 4,950. Everything above the diagonal: $(25 - 5) \div 2 = 10$, and $(100 \times 100 - 100) \div 2 = 4,950$.

4.2 B. *sat* comes after *cat*, so *cat* may not look ahead at it. A and C look backwards; D is a token looking at itself, which is allowed.

4.3 The last row (the final *the*). No token comes after it, so it may look at every token, itself included.

4.4 False. $e^0 = 1$, so a score of 0 would still get a real share of the weight. Only $-\infty$ gives $e^{-\infty} = 0$.

5. Softmax, row by row · rewatch ► 1:20-1:31

5.1 0.14 and 0.86. Total $0.217 + 1.284 = 1.501$; $0.217 \div 1.501 = 0.14$ and $1.284 \div 1.501 = 0.86$, as in the video.

5.2 It has only one unmasked entry. $e^s \div e^s = 1$ for its own score, and every masked entry gives $e^{-\infty} = 0$.

5.3 B. $e^{s+1} = e \times e^s$, and the common factor e cancels in the division: only differences between scores matter (F07).

5.4 False. $e^x > 0$ for every x : in row *sat*, the score -0.38 becomes the weight 0.28 .

6. Mixing the values · rewatch ► 1:31-1:43

6.1 0.27. $-0.003 - 0.049 + 0.322 = 0.270 \approx 0.27$, the video's value.

6.2 B. $(5 \times 5) \cdot (5 \times 4) = 5 \times 4$: one new 4-number vector per token, the same shape as X .

6.3 sat's (0.74); the final the's (weight 0). The weights set each value's share. The final *the* comes after *on*, so it is masked.

6.4 True. The first token's only weight is 1.00 , on itself: $1.00 \cdot v_{\text{The}} = v_{\text{The}}$.

7. The whole formula, in NumPy · rewatch ► 1:43-2:16

7.1 project $\rightarrow Q \cdot K^T \rightarrow \div \sqrt{d} \rightarrow \text{mask} \rightarrow \text{softmax} \rightarrow \text{multiply by } V$. Scores before weights, the mask before the softmax (so masked entries become 0), values last.

7.2 A. GPUs are built to do huge matrix multiplications in parallel, and that is exactly what attention is.

7.3 (a) (1, 0) · (b) weights 0.33 and 0.67, output (0.33, 0.67) · (c) rows 1 and 2 change, to (0.8, 0.6) and (0.6, 0.8); row 3 stays (0.75, 0.75) The printed rows are $[1. \ 0.]$, $[0.33 \ 0.67]$ and $[0.75 \ 0.75]$. (a) Token 1 sees only itself, so it gets its own value, $(1, 0)$. (b) Scores $q_2 \cdot k_1 = 0$ and $q_2 \cdot k_2 = 1$; divided by $\sqrt{2} \approx 1.41$ they are 0 and 0.71 . $e^0 = 1$ and $e^{0.71} \approx 2.03$, so the weights are $1 \div 3.03 = 0.33$ and $2.03 \div 3.03 = 0.67$, and the output is $0.33 \cdot (1, 0) + 0.67 \cdot (0, 1) = (0.33, 0.67)$. (c) Without the mask, tokens 1 and 2 also mix in the values of later tokens. Token 3 is last: it already saw everything, so its row does not change.